

app\temperature_simulation.py

```

1  # -*- coding: utf-8 -*-
2  """
3  @file      temperature_simulation.py
4  @brief     Module de calcul de température simulée selon formule prédictive.
5  @details   Ce module implémente la formule de simulation de température:
6              $T_{sim} = T_{mes} + ((T_{prev} - T_{mes}) * n * \exp(k_m))$ 
7             Inclut la validation complète des paramètres, la gestion
8             des erreurs numériques et l'exécution de simulations complètes.
9  @author    Léo Mendes
10 @project    2510_RegulationsChauffage
11 @Mandant    Oblo_solution
12 @date       2025-09-18
13 @version    1.0.0
14 @copyright  Copyright (c) 2025
15 """
16
17 # Import des modules système standard
18 import math
19 # Import des annotations de type pour la documentation
20 from typing import Optional
21
22
23 # -----
24 # Fonctions de calcul principal
25 # -----
26
27 def calculate_simulated_temperature(t_mes: float, t_prev: float, n: float, k_m: float) -> Optional[float]:
28     """
29     @brief   Calcule la température simulée selon la formule prédictive.
30     @details Implémente la formule  $T_{sim} = T_{mes} + ((T_{prev} - T_{mes}) * n * \exp(k_m))$ 
31             avec validation complète des paramètres et gestion des erreurs
32             numériques (overflow, underflow, valeurs infinies).
33
34     @param t_mes    Température mesurée [°C].
35     @param t_prev   Température prévue [°C].
36     @param n        Paramètre de pondération [-∞, +∞].
37     @param k_m      Paramètre exposant pour fonction exponentielle [-∞, +∞].
38
39     @return         Température simulée [°C] ou None en cas d'erreur.
40
41     @exception      OverflowError si exp(k_m) dépasse les limites numériques.
42     @warning         Limitation automatique des paramètres extrêmes.
43     """
44     try:
45         # Validation du type des paramètres d'entrée
46         if not all(isinstance(param, (int, float)) for param in [t_mes, t_prev, n, k_m]):
47             # Affichage d'erreur si types incorrects
48             print("[CALC] Erreur: tous les paramètres doivent être des nombres")
49             return None
50
51         # Vérification des plages de températures raisonnables pour mesure
52         if not (-100.0 <= t_mes <= 200.0):
53             # Avertissement pour température mesurée hors plage normale
54             print(f"[CALC] Attention: température mesurée hors plage normale: {t_mes}°C")
55
56         # Vérification des plages de températures raisonnables pour prévision
57         if not (-100.0 <= t_prev <= 200.0):
58             # Avertissement pour température prévue hors plage normale
59             print(f"[CALC] Attention: température prévue hors plage normale: {t_prev}°C")
60
61         # Limitation du paramètre k_m pour éviter l'overflow exponentiel
62         if abs(k_m) > 50:
63             # Erreur critique si k_m trop élevé
64             print(f"[CALC] Erreur: paramètre k_m trop élevé: {k_m}")
65             return None
66
67         # Calcul de l'exponentielle avec gestion des cas d'overflow
68         try:
69             # Application de la fonction exponentielle
70             exp_km = math.exp(k_m)
71         except OverflowError:
72             # Gestion de l'overflow lors du calcul exponentiel
73             print(f"[CALC] Erreur: overflow lors du calcul de exp({k_m})")
74             return None
75
76         # Vérification que l'exponentielle reste dans une plage numérique raisonnable
77         if exp_km > 1e10:
78             # Limitation de la valeur exponentielle pour éviter instabilité
79             print(f"[CALC] Erreur: exp(k_m) = {exp_km} trop élevé")
80             return None
81
82         # Calcul de la différence entre température prévue et mesurée
83         temp_diff = t_prev - t_mes
84

```

```
85     # Application de la formule complète de simulation
86     t_sim = t_mes + (temp_diff * n * exp_km)
87
88     # Vérification que le résultat est un nombre fini valide
89     if not math.isfinite(t_sim):
90         # Erreur si résultat infini ou NaN
91         print(f"[CALC] Erreur: résultat non fini: {t_sim}")
92         return None
93
94     # Limitation du résultat dans une plage physiquement raisonnable
95     if not (-200.0 <= t_sim <= 500.0):
96         # Avertissement pour température simulée hors plage physique
97         print(f"[CALC] Attention: température simulée hors plage raisonnable: {t_sim}°C")
98
99     # Affichage de confirmation avec détail du calcul effectué
100    print(f"[CALC] Calcul réussi: T_sim = {t_mes} + (({t_prev} - {t_mes}) * {n} * exp({k_m})) = {t_sim:.2f}°C")
101    # Retour de la température simulée calculée
102    return t_sim
103
104    except Exception as e:
105        # Gestion d'exception générale pour erreurs inattendues
106        print(f"[CALC] Erreur inattendue lors du calcul: {e}")
107        return None
108
109    def validate_simulation_parameters(probe_type: str, n: float, k_m: float, t_mes: float, t_prev: float) -> bool:
110        """
111        @brief Valide tous les paramètres de simulation avant exécution.
112        @details Fonction de validation complète qui vérifie la cohérence
113               et les plages de tous les paramètres d'entrée. Inclut
114               la vérification interactive pour les écarts importants.
115
116        @param probe_type Type de sonde à utiliser pour la simulation.
117        @param n Paramètre de pondération de la simulation.
118        @param k_m Paramètre exposant de la fonction exponentielle.
119        @param t_mes Température mesurée à valider [°C].
120        @param t_prev Température prévue à valider [°C].
121
122        @return True si tous les paramètres sont valides.
123
124        @warning Demande confirmation utilisateur pour écarts importants.
125        """
126        # Affichage du début de la procédure de validation
127        print("[VALIDATION] Vérification des paramètres de simulation...")
128
129        # Liste des types de sondes supportés par le système
130        valid_probe_types = ["PT1000", "Ni1000_TK5000", "NTC_10k", "Ni1000 TK5000", "NTC 10k 3977"]
131        # Vérification que le type de sonde est dans la liste supportée
132        if probe_type not in valid_probe_types:
133            # Affichage d'erreur avec liste des types supportés
134            print(f"[VALIDATION] Type de sonde non supporté: {probe_type}")
135            print(f"[VALIDATION] Types supportés: {valid_probe_types}")
136            return False
137        else:
138            # Confirmation que le type de sonde est valide
139            print(f"[VALIDATION] Type de sonde valide: {probe_type}")
140
141        # Vérification du paramètre n dans sa plage de fonctionnement recommandée
142        if not (-2.0 <= n <= 2.0):
143            # Erreur si paramètre n hors plage acceptable
144            print(f"[VALIDATION] Paramètre n hors plage [-2.0, 2.0]: {n}")
145            return False
146        else:
147            # Confirmation de validité du paramètre n
148            print(f"[VALIDATION] Paramètre n valide: {n}")
149
150        # Vérification du paramètre k_m dans sa plage de sécurité numérique
151        if not (-10.0 <= k_m <= 10.0):
152            # Erreur si k_m risque de causer overflow exponentiel
153            print(f"[VALIDATION] Paramètre k_m hors plage [-10.0, 10.0]: {k_m}")
154            return False
155        else:
156            # Confirmation de validité du paramètre k_m
157            print(f"[VALIDATION] Paramètre k_m valide: {k_m}")
158
159        # Vérification de la température mesurée dans plage physique normale
160        if not (-50.0 <= t_mes <= 150.0):
161            # Erreur si température mesurée hors plage d'application
162            print(f"[VALIDATION] Température mesurée hors plage [-50°C, 150°C]: {t_mes}°C")
163            return False
164        else:
165            # Confirmation de validité de la température mesurée
166            print(f"[VALIDATION] Température mesurée valide: {t_mes}°C")
167
168        # Vérification de la température prévue dans plage physique normale
169        if not (-50.0 <= t_prev <= 150.0):
170            # Erreur si température prévue hors plage d'application
171            print(f"[VALIDATION] Température prévue hors plage [-50°C, 150°C]: {t_prev}°C")
```

```

172         return False
173     else:
174         # Confirmation de validité de la température prévue
175         print(f"[VALIDATION] Température prévue valide: {t_prev}°C")
176
177     # Calcul de l'écart absolu entre températures pour vérification cohérence
178     temp_diff = abs(t_prev - t_mes)
179     # Vérification si l'écart entre températures est anormalement important
180     if temp_diff > 50.0:
181         # Avertissement pour écart important entre températures
182         print(f"[VALIDATION] Écart important entre températures: {temp_diff:.1f}°C")
183         print(f"[VALIDATION] T_mes={t_mes}°C, T_prev={t_prev}°C")
184
185         # Procédure de demande de confirmation utilisateur pour écart important
186         try:
187             # Demande de confirmation interactive
188             confirm = input("[VALIDATION] Continuer malgré l'écart important? (o/N): ").strip().lower()
189             # Vérification de la réponse utilisateur
190             if confirm != 'o' and confirm != 'oui':
191                 # Annulation si utilisateur refuse de continuer
192                 print("[VALIDATION] Validation annulée par l'utilisateur")
193                 return False
194         except KeyboardInterrupt:
195             # Gestion de l'interruption par Ctrl+C
196             print("\n[VALIDATION] Validation interrompue")
197             return False
198
199     # Confirmation finale que tous les paramètres sont validés
200     print("[VALIDATION] Tous les paramètres sont valides")
201     return True
202
203 def run_temperature_simulation(simulation_data: dict) -> Optional[float]:
204     """
205     @brief Exécute une simulation complète de température avec dictionnaire.
206     @details Fonction principale d'orchestration qui extrait les paramètres
207             d'un dictionnaire, valide tous les paramètres et exécute la
208             simulation complète avec gestion d'erreur robuste.
209
210     @param simulation_data Dictionnaire contenant tous les paramètres requis:
211             - probe_type: Type de sonde (str)
212             - n: Paramètre de pondération (float)
213             - k_m: Paramètre exposant (float)
214             - temperature: Température mesurée (float)
215             - forecast_temperature: Température prévue (float)
216
217     @return Température simulée [°C] ou None si erreur.
218
219     @warning Retourne None pour toute erreur de validation ou calcul.
220     """
221     # Affichage d'en-tête de la simulation avec séparateur visuel
222     print("\n=== SIMULATION DE TEMPÉRATURE ===")
223
224     # Bloc de traitement avec gestion complète des exceptions
225     try:
226         # Extraction sécurisée de tous les paramètres du dictionnaire
227         probe_type = simulation_data["probe_type"]
228         n = float(simulation_data["n"])
229         k_m = float(simulation_data["k_m"])
230         t_mes = float(simulation_data["temperature"])
231         t_prev = float(simulation_data["forecast_temperature"])
232
233         # Affichage formaté de tous les paramètres extraits
234         print(f"[SIM] Paramètres d'entrée:")
235         print(f"[SIM] Type de sonde: {probe_type}")
236         print(f"[SIM] Paramètre n: {n}")
237         print(f"[SIM] Paramètre k_m: {k_m}")
238         print(f"[SIM] Température mesurée: {t_mes}°C")
239         print(f"[SIM] Température prévue: {t_prev}°C")
240
241         # Étape de validation complète des paramètres extraits
242         if not validate_simulation_parameters(probe_type, n, k_m, t_mes, t_prev):
243             # Arrêt immédiat si validation échoue
244             print("[SIM] Échec de la validation des paramètres")
245             return None
246
247         # Calcul de la température simulée avec paramètres validés
248         t_sim = calculate_simulated_temperature(t_mes, t_prev, n, k_m)
249
250         # Vérification que le calcul a réussi
251         if t_sim is None:
252             # Gestion du cas d'échec de calcul
253             print("[SIM] Échec du calcul de la température simulée")
254             return None
255
256         # Section d'affichage des résultats avec formatage soigné
257         print(f"\n[SIM] RÉSULTAT:")
258         print(f"[SIM] Température simulée: {t_sim:.2f}°C")

```

```
259
260     # Analyse comparative automatique du résultat obtenu
261     ecart_mesure = abs(t_sim - t_mes)
262     ecart_prevision = abs(t_sim - t_prev)
263
264     # Classification intelligente du résultat selon les écarts
265     if ecart_mesure < 0.1:
266         # Résultat très proche de la mesure actuelle
267         print(f"[SIM] Simulation proche de la mesure (écart: {ecart_mesure:.3f}°C)")
268     elif ecart_prevision < ecart_mesure:
269         # Résultat tend vers la prévision plutôt que la mesure
270         print(f"[SIM] Simulation tend vers la prévision")
271     else:
272         # Résultat tend vers la mesure plutôt que la prévision
273         print(f"[SIM] Simulation tend vers la mesure")
274
275     # Retour du résultat calculé et validé
276     return t_sim
277
278 except KeyError as e:
279     # Gestion des paramètres manquants dans le dictionnaire
280     print(f"[SIM] Paramètre manquant: {e}")
281     return None
282
283 except (ValueError, TypeError) as e:
284     # Gestion des erreurs de conversion de type
285     print(f"[SIM] Erreur de conversion de type: {e}")
286     return None
287
288 except Exception as e:
289     # Capture de toute autre exception non prévue
290     print(f"[SIM] Erreur inattendue: {e}")
291     return None
```